Intune Custom Compliance & Configuration Items (Scripted)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Intune Custom Compliance & Configuration Items (Scripted) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Intune Custom Compliance & Configuration Items (Scripted) 5

What custom compliance is . . . 5

Scripted configuration items . . . 5

How it works . . . 5

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Intune Custom Compliance & Configuration Items (Scripted) 6

2.1 Licensing . . . 6

2.2 Device enrollment . . . 7

2.3 Supported Windows OS . . . 7

2.4 A compliance baseline (built-in policy) . . . 7

2.5 PowerShell script signing / execution . . . 7

2.6 Target device groups . . . 7

2.7 Conditional Access (to enforce, not just report) . . . 7

Checklist . . . 7

Global Requirements — Intune Solutions (category-level) 8

G1. Microsoft Entra ID tenant . . . 8

G2. Microsoft Intune subscription . . . 8

G3. Administrative roles . . . 8

G4. Device enrollment in Intune . . . 8

G5. Pilot security group . . . 8

G6. Supported Windows baseline . . . 9

G7. Network access to Microsoft cloud endpoints . . . 9

3. Implementation Guide — Intune Custom Compliance & Configuration Items (Scripted) 9

Step 1 — Create the scope groups . . . 9

Step 2 — Author the detection script . . . 10

Step 3 — Author the JSON rule file . . . 10

Step 4 — Add scripted configuration items (set the state) . . . 10

Step 5 — Upload the detection script . . . 11
Step 6 — Create the custom compliance policy . . . 11
Step 7 — Require compliance in Conditional Access . . . 11
Step 8 — Apply on a pilot device . . . 11
Step 9 — Validate the pilot . . . 12
Step 10 — Expand to production . . . 12
5. Testing Guide — Intune Custom Compliance & Configuration Items (Scripted) 12
Test 1 — Script + rule pairing is valid . . . 12
Test 2 — Detection script runs on device . . . 12
Test 3 — Config item applied . . . 12
Test 4 — Custom compliance evaluates . . . 13
Test 5 — A failing rule turns the device noncompliant . . . 13
Test 6 — Conditional Access effect . . . 13
Test 7 — Fleet report . . . 13
Pass/fail gate . . . 13
6. Troubleshooting Guide — Intune Custom Compliance & Configuration Items (Scripted) 13
A. Custom compliance shows Error (not pass/fail) . . . 13
B. Rule always fails (or always passes) . . . 14
C. Config item not taking effect . . . 14
D. Compliance state not updating . . . 14
E. Conditional Access not blocking / over-blocking . . . 14
F. Diagnostics quick reference . . . 15
G. When to escalate . . . 15
7. Best Practices — Intune Custom Compliance & Configuration Items (Scripted) 15
8. Deliverables Summary — Intune Custom Compliance & Configuration Items (Scripted) 16
Documentation . . . 16
PowerShell scripts . . . 16
What the buyer receives . . . 16
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — Intune Custom Compliance & Configuration Items (Scripted)

White-label · Intune Solutions

Intune's built-in compliance settings cover the basics — encryption, OS version, password — but real security standards go further: a specific registry value, a minimum app version, a control the GUI simply can't reach. **Custom compliance** closes that gap. It pairs a **PowerShell detection script** with a **JSON rule** that Intune evaluates on the device, then folds the result into device compliance — and, through Conditional Access, into access decisions. For the settings templates can't express, **scripted configuration items** (settings-catalog custom settings) push exactly the value you need.

This is a complete, deploy-it-today package for authoring, deploying, and managing scripted custom compliance and configuration items with Microsoft Intune, written for an administrator with no prior custom-compliance experience.

What you get

- A plain-language Solution Overview of how custom compliance works (detection script + JSON rule + settings mapping), scripted configuration items, and how compliance feeds Conditional Access.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered Intune steps — including a worked detection-script and JSON-rule example you can adapt.
- Five PowerShell scripts for discovery, rule/JSON validation, fleet custom-compliance reporting, safe remediation, and building the compliance-scope group.
- Testing and Troubleshooting guides for a confident rollout.

What this delivers

A repeatable authoring model: write the detection script, author the JSON discovery/rule file and settings mapping, upload it as a custom compliance policy, add any scripted configuration items, assign to a pilot, validate evaluation and the Conditional Access effect, then expand — so device state is measured and enforced exactly the way your standards require.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Device state measured and enforced to your standards — custom detection scripts and scripted configuration items covering what the built-in settings can't, wired into Conditional Access, and proven on a pilot before you scale.

1. Solution Overview — Intune Custom Compliance & Configuration Items (Scripted)

What custom compliance is

Intune custom compliance lets you evaluate device state that the built-in compliance settings can't reach — a registry value, a running service, an installed app version, a security control specific to your organization. It works by pairing two artifacts:

1. A PowerShell **detection script** that runs on the device and returns a small JSON payload of the values it discovered.
1. A **JSON rule file** that tells Intune how to judge those values (operators, thresholds) and what remediation message to show the user when a value fails.

You upload the detection script under `Devices → Compliance → Scripts`, then reference it in a custom compliance policy created under `Devices → Compliance → Create policy` (custom compliance). Intune runs the script, reads the returned settings, applies the JSON rule, and marks the device compliant or noncompliant on those custom settings — alongside the built-in rules.

Scripted configuration items

Some device state must be **set**, not just measured. Where the configuration templates and the built-in compliance UI can't express a setting, use **scripted configuration items** — custom settings authored in the settings catalog under `Devices → Configuration`. These push the exact registry value, policy, or OMA-URI you need, so the device reaches the state your custom compliance rule then verifies. Together they form a measure-and-set pair: the config item makes the device correct; custom compliance proves it.

How it works

1. You author a detection script that outputs a JSON object of named settings (for example a registry value or an app version).
1. You author a JSON rule file that maps each setting to an operator and value, with a per-rule remediation message and severity.
1. You upload the detection script (`Devices → Compliance → Scripts`) and create a custom compliance policy that references it and imports the JSON rule.
1. You assign the policy to a pilot group; Intune runs the script on each device and evaluates the rule.
1. Non-matching settings mark the device noncompliant on the custom rule; the user sees your remediation message.
1. A Conditional Access policy that requires a compliant device then gates access — so a failed

custom check can block access, not just report.

What you can control

• Any device-side value a PowerShell script can read (registry, WMI/CIM, file/app version, service state, certificate presence) evaluated with your own operators and thresholds.

• Per-rule remediation messages, severity, and the JSON discovery/rule mapping.

• Scripted configuration items for settings the templates can't reach.

• How custom compliance participates in overall device compliance and Conditional Access.

Benefits

• **Reach** — measure and enforce controls the built-in settings can't express.

• **Precision** — your own operators, thresholds, and messages per rule.

• **Enforcement** — custom results feed device compliance and Conditional Access, not just reports.

• **Repeatability** — a documented author → upload → assign → validate model reused for every check.

Limitations (be honest with buyers)

• Detection scripts run in **64-bit PowerShell** on the device and must emit valid JSON matching the rule file; a script or JSON error surfaces as an error state, not a pass.

• Custom compliance evaluates **Windows** devices; other platforms use their own compliance settings.

• The device must be enrolled and healthy for the script to run and report.

• Scripts should be **signed** (or execution policy allowed) per your standards; treat detection scripts as code you review and control.

The standard deployment model

> ***Author the detection script + JSON rule → add scripted config items → upload + create the custom compliance policy → assign to a pilot → validate evaluation and the Conditional Access effect → expand.*** *repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Intune Custom Compliance & Configuration Items (Scripted)

Verify **every** requirement before authoring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

2.1 Licensing

Description: Intune (Plan 1). Custom compliance and configuration items are included features. Why: Intune hosts the detection script, custom compliance policy, and configuration items. Verify: M365 admin center → `Billing → Licenses`. Implement: Assign M365 E3/E5 or Business Premium (Intune included).

2.2 Device enrollment

Description: Targeted devices enrolled in Intune. Why: Detection scripts and config items apply only to enrolled devices. Verify: Intune → `Devices → All devices`. Implement: Enable automatic enrollment; enroll devices.

2.3 Supported Windows OS

Description: Windows 10/11 (Pro/Enterprise/Education), current build. Why: Custom compliance detection scripts run in 64-bit PowerShell on a supported Windows build. Verify: `winver`; run `CUSTCOMP-Discovery.ps1`. Implement: Patch/upgrade to a supported build.

2.4 A compliance baseline (built-in policy)

Description: A standard device compliance policy already assigned (encryption, OS version, etc.). Why: Custom compliance extends the baseline; it is not a replacement for the built-in rules. Verify: Intune → `Devices → Compliance`; a Windows policy exists and is assigned. Implement: Create a built-in compliance policy first (see the Device Compliance Policies solution).

2.5 PowerShell script signing / execution

Description: Detection scripts are trusted code — signed, or execution allowed per your standard. Why: The device must run the detection script; unsigned/blocked scripts fail to evaluate. Verify: `Get-ExecutionPolicy -List`; confirm scripts you author can run. Implement: Sign scripts with your code-signing certificate, or set an appropriate execution policy via Intune.

2.6 Target device groups

Description: A pilot group and a broad group for staged assignment. Why: Every custom check is piloted before the fleet gets it. Verify: Entra → `Groups`; the pilot/broad groups exist. Implement: Run `CUSTCOMP-Set-ScopeTargeting.ps1`, or Entra → `Groups → New group`.

2.7 Conditional Access (to enforce, not just report)

Description: A Conditional Access policy that requires a compliant device. Why: Custom compliance only gates access when Conditional Access requires compliance. Verify: Entra → `Security → Conditional Access`; a "require compliant device" policy exists. Implement: Create a Conditional Access policy requiring a compliant device (start in report-only).

Checklist

- Intune licensing assigned

- Devices enrolled
- Windows 10/11 supported build — `CUSTCOMP-Discovery.ps1`
- A built-in compliance baseline is assigned
- PowerShell script signing / execution allowed
- Pilot + broad Entra groups created — `CUSTCOMP-Set-ScopeTargeting.ps1`
- Conditional Access "require compliant device" policy (report-only to start)

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Intune Custom Compliance & Configuration Items (Scripted)

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`).

> ***Model:*** *Author a detection script and a JSON rule for the check the built-in settings can't reach, add any scripted configuration items to set the state, upload and create a custom compliance policy, assign to a pilot, validate evaluation and the Conditional Access effect, then expand.*

Name every artifact `<Purpose> – <Scope> – <Stage>` and reuse the name verbatim across the policy, the group, and the docs. This solution uses these names:

Artifact	Name
Detection script	`RegAppCheck – Custom Compliance – Pilot`
JSON rule file	`RegAppCheck-Rule – Custom Compliance – Pilot.json`
Custom compliance policy	`Custom Compliance – Windows – Pilot`
Scripted config item	`SecuritySetting – Config Item – Pilot`
Pilot device group	`CustomCompliance – Devices – Pilot`

Step 1 — Create the scope groups

1. Entra → `Groups` → `New group` → Security → `CustomCompliance – Devices – Pilot` (a few

representative devices). Create. (Or run `CUSTCOMP-Set-ScopeTargeting.ps1`.)

1. Create `CustomCompliance – Devices – Broad` (the rest of the fleet). Create.

Step 2 — Author the detection script

The detection script runs on the device and must output a **single-line JSON object** whose property names match the setting names your rule file references. Here is a worked example that checks a registry value and an app version:

```
# RegAppCheck – Custom Compliance – Pilot  (detection script)
$reg = (Get-ItemProperty 'HKLM:\SOFTWARE\Contoso\Security' -Name 'HardeningLevel' -ErrorAction SilentlyCon
$app = (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*' -ErrorAction Silent
        Where-Object { $_.DisplayName -eq 'Contoso Agent' } | Select-Object -First 1).DisplayVersion
$out = @{
    HardeningLevel = [int]($reg  | ForEach-Object { $_ })
    AgentVersion   = [string]($app | ForEach-Object { $_ })
}
return $out | ConvertTo-Json -Compress
```

Save it as a `.ps1` file. The script emits, for example, `{"HardeningLevel":2,"AgentVersion":"4.1.0"}`.

Step 3 — Author the JSON rule file

The rule file tells Intune how to judge each setting the script returned, and the message the user sees when a rule fails. It matches the property names from Step 2:

```
{
  "Rules": [
    {
      "SettingName": "HardeningLevel",
      "Operator": "GreaterEquals",
      "DataType": "Int64",
      "Operand": 2,
      "MoreInfoUrl": "https://intranet.contoso.com/hardening",
      "RemediationStrings": [
        { "Language": "en_US",
          "Title": "Security hardening not applied",
          "Description": "This device must be at hardening level 2 or higher. Apply the security config it
      ]
    },
    {
      "SettingName": "AgentVersion",
      "Operator": "GreaterEquals",
      "DataType": "Version",
      "Operand": "4.1.0",
      "MoreInfoUrl": "https://intranet.contoso.com/agent",
      "RemediationStrings": [
        { "Language": "en_US",
          "Title": "Contoso Agent out of date",
          "Description": "Update Contoso Agent to 4.1.0 or later, then sync." }
      ]
    }
  ]
}
```

Validate the pairing before upload: run `CUSTCOMP-Validation.ps1 -ScriptPath <script.ps1> -RulePath <rule.json>`.

Step 4 — Add scripted configuration items (set the state)

For state that must be **set** (so the rule above can pass), author a scripted configuration item.

1. Go to `Devices` → `Configuration` → `Create` → `New policy`.

2. Platform: Windows 10 and later. Profile type: Settings catalog. Create.

3. Name: `SecuritySetting – Config Item – Pilot`.

4. Add the setting the templates can't reach (search the settings catalog, or use a custom OMA-URI / registry setting that writes `HardeningLevel`). Set the required value.

1. Assignments → Add groups → `CustomCompliance – Devices – Pilot`. Create.

Step 5 — Upload the detection script

1. Go to `Devices → Compliance → Scripts → Add`.

2. Name: `RegAppCheck – Custom Compliance – Pilot`. Next.

3. Paste the detection script from Step 2. Set `Run this script using the logged-on credentials` = No; `Enforce script signature check` per your standard; `Run script in 64-bit PowerShell` = Yes. Next.

1. Review + add.

Step 6 — Create the custom compliance policy

1. Go to `Devices → Compliance → Create policy`.

2. Platform: Windows 10 and later. Create.

3. Name: `Custom Compliance – Windows – Pilot`. Next.

4. On Compliance settings → `Custom Compliance` → set `Custom compliance` = Require, choose the uploaded script `RegAppCheck – Custom Compliance – Pilot`, and upload the JSON rule file from Step 3. Next.

1. Actions for noncompliance: keep `Mark device noncompliant` (add a grace period if desired). Next.

2. Assignments → Add groups → `CustomCompliance – Devices – Pilot`. Next.

3. Review + create.

Step 7 — Require compliance in Conditional Access

1. Entra → `Security → Conditional Access → New policy` (or use the existing require-compliant policy).

2. Grant → `Require device to be marked as compliant`. Start in `Report-only` mode for the pilot users. Save.

Step 8 — Apply on a pilot device

1. On a pilot device, sync policy (`Settings → Accounts → Access work or school → Info → Sync`).
2. Confirm the config item applied and the detection script ran, or run `CUSTCOMP-Report.ps1`.

Step 9 — Validate the pilot

1. Run `CUSTCOMP-Discovery.ps1` on a pilot device → confirms the device-side values the script reads.
2. Run `CUSTCOMP-Report.ps1` → confirms custom-compliance status across the pilot group (Graph).
3. Confirm a device that fails a rule shows the remediation message and turns noncompliant; confirm the Conditional Access report-only result reflects it. Complete the Testing Guide.

Step 10 — Expand to production

1. Assign the detection script, config item, and custom compliance policy to `CustomCompliance – Devices – Broad`; roll out in waves.
1. Move the Conditional Access policy from `Report-only` to `On` once coverage is confirmed.
2. Re-validate each wave with `CUSTCOMP-Report.ps1`.

Order of operations

`Scope groups → Detection script → JSON rule → Config item → Upload script → Custom compliance policy → CA require-compliant → Apply → Validate → Expand`

5. Testing Guide — Intune Custom Compliance & Configuration Items (Scripted)

Run after the pilot policy applies (Implementation Step 9) and after any change to the detection script, JSON rule, or config item.

Test 1 — Script + rule pairing is valid

Do: On your workstation run `CUSTCOMP-Validation.ps1 -ScriptPath <script.ps1> -RulePath <rule.json>`. Expected: Reports valid JSON, and every setting name in the rule is produced by the script (no mismatch).

Test 2 — Detection script runs on device

Do: On a pilot device run `CUSTCOMP-Discovery.ps1`. Expected: The device-side values the detection script reads are shown (registry value, app version) as valid JSON.

Test 3 — Config item applied

Do: Intune → `Devices` → `Configuration` → `SecuritySetting – Config Item – Pilot` → Device status. Expected: Pilot devices show Succeeded; the target setting is present on the device.

Test 4 — Custom compliance evaluates

Do: Intune → `Devices` → `Compliance` → `Custom Compliance – Windows – Pilot` → Device status. Expected: Pilot devices show a custom-compliance result (compliant when the rule passes).

Test 5 — A failing rule turns the device noncompliant

Do: On a test device, make one checked value fail (e.g. remove the setting), sync, and re-evaluate. Expected: The device turns noncompliant on the custom rule and the user sees the remediation message.

Test 6 — Conditional Access effect

Do: With the CA policy in Report-only, review `Sign-in logs` → `Conditional Access` for a noncompliant device. Expected: The report-only result shows the "require compliant device" grant would block that device.

Test 7 — Fleet report

Do: Run `CUSTCOMP-Report.ps1`. Expected: A per-device custom-compliance status across the assigned group.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–4 pass; Test 5 shows a failing rule turns the device noncompliant with the message
Broad pass	Tests 4, 6, 7 pass; the Conditional Access effect is confirmed before moving CA to On

6. Troubleshooting Guide — Intune Custom Compliance & Configuration Items (Scripted)

Use `CUSTCOMP-Discovery.ps1` for device-side values, `CUSTCOMP-Validation.ps1` for the script/rule pairing, `CUSTCOMP-Report.ps1` for fleet status, and `CUSTCOMP-Remediation.ps1` for a safe sync.

A. Custom compliance shows Error (not pass/fail)

Symptom	Cause	Remediation
Device shows Error on the custom rule	Detection script threw, or returned invalid JSON	Run `CUSTCOMP-Discovery.ps1`; confirm the script returns valid single-line JSON; run `CUSTCOMP-Validation.ps1`
"Setting not found" style result	Rule setting name not emitted by the script	Align the rule `SettingName` values with the script's JSON property names
Script never runs	Signature check on, script unsigned; or not 64-bit	Sign the script or adjust the signature setting; ensure `Run in 64-bit PowerShell` = `Yes`

B. Rule always fails (or always passes)

Symptom	Cause	Remediation
Value looks right but rule fails	Wrong `DataType`/`Operator` (e.g. Version vs String)	Match the rule `DataType` to what the script returns; use `Version` for app versions
Rule passes when it shouldn't	Script returns null/empty and the operator tolerates it	Have the script return a definite value; test with a device that should fail

C. Config item not taking effect

• Confirm `SecuritySetting – Config Item – Pilot` shows Succeeded under Device status; the setting the

custom rule checks must be applied by the config item (or otherwise present) before the rule can pass.

• Sync the device (`CUSTCOMP-Remediation.ps1`) and allow a policy cycle before re-evaluating.

D. Compliance state not updating

• Custom compliance re-evaluates on the device's schedule; force a sync with `CUSTCOMP-Remediation.ps1`

and wait a policy cycle. Confirm the device is enrolled, checked in, and in the assigned group.

E. Conditional Access not blocking / over-blocking

• If a noncompliant device is not blocked, confirm the Conditional Access policy is On (not Report-only) and targets the right users, and that it requires a compliant device.

• If devices are blocked unexpectedly, move the CA policy back to Report-only and fix the failing rule first; a custom-rule Error counts as noncompliant.

F. Diagnostics quick reference

```
Get-ComputerInfo | Select WindowsProductName, OsBuildNumber, WindowsVersion
Get-ItemProperty 'HKLM:\SOFTWARE\Contoso\Security' -EA SilentlyContinue   # the checked registry value
Get-ExecutionPolicy -List                                                 # signing/execution posture
Start-Process "$env:WINDIR\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM"   # MDM sync
```

IME logs (device-side script/compliance execution): `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs`.

G. When to escalate

If the script returns valid JSON locally, the rule pairing validates, the config item shows Succeeded, and the device is synced and in the assigned group but the custom result still shows Error after a policy cycle, collect the IntuneManagementExtension logs and open a Microsoft support case.

7. Best Practices — Intune Custom Compliance & Configuration Items (Scripted)

• **Pilot every check.** Assign a new detection script + rule to a small pilot group first — a bad rule or a script error can turn devices noncompliant fleet-wide. This is the single most important practice.

• **Keep the script and rule in lockstep.** Every setting name in the JSON rule must be emitted by the detection script. Validate the pairing with `CUSTCOMP-Validation.ps1` before every upload.

• **Emit compact, valid JSON.** Have the detection script return one JSON object with the correct data types (int, string, version); a malformed payload reads as an error, not a pass.

• **Measure and set together.** Pair a scripted configuration item that sets the state with a custom rule that verifies it — so devices are made correct, then proven correct.

• **Write clear remediation messages.** The user-facing title/description in the rule is what drives self-remediation; make it specific and actionable (what to fix, then sync).

• **Sign your scripts.** Treat detection scripts as reviewed, controlled code; sign them or scope execution per your standard.

• **Enforce through Conditional Access — carefully.** Require a compliant device, but start in Report-only and confirm the effect before turning it On, so a rule mistake can't lock users out.

• **Name artifacts consistently.** Use `<Purpose> – <Scope> – <Stage>` for the script, rule, policy, config item, and group, and reuse the names verbatim for auditability.

• **Version your rules.** Record the detection script and JSON rule content per revision so you can roll back a change that misfires.

• **Watch the report.** Track custom-compliance status with `CUSTCOMP-Report.ps1` and chase devices stuck in error before they age into a Conditional Access block.

8. Deliverables Summary — Intune Custom Compliance & Configuration Items (Scripted)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How custom compliance works (detection script + JSON rule + settings mapping), scripted config items, and the Conditional Access link
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps with a worked detection-script + JSON-rule example
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`CUSTCOMP-Discovery.ps1`	Reports OS build and the device-side values a detection script reads (registry value, app version) as JSON; read-only
`CUSTCOMP-Validation.ps1`	Validates a detection script + JSON rule pairing (valid JSON, setting names match); read-only
`CUSTCOMP-Report.ps1`	Fleet-wide custom-compliance status per device (Microsoft Graph); read-only
`CUSTCOMP-Remediation.ps1`	Safe remediation; forces an MDM sync and re-evaluation (default `-WhatIf`)
`CUSTCOMP-Set-ScopeTargeting.ps1`	Creates/populates the custom-compliance pilot/broad device groups (Microsoft Graph)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.

• A repeatable process following the repeatable model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Device state measured and enforced to your standards — scripted detection + JSON rules and config items covering what the built-in settings can't, wired into Conditional Access, and proven on a pilot.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CUSTCOMP-Discovery.ps1

CUSTCOMP-Remediation.ps1

Purpose: Triggers an Intune/MDM sync so the custom compliance detection script and config item re-apply and the device re-reports its state. Defaults to -WhatIf: it shows what it would do and changes nothing until you pass -WhatIf:$false (or -Confirm:$false). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS In preview, the actions it would take. When applied, an MDM sync is requested; re-run CUSTCOMP-Report.ps1 / check the portal after a policy cycle. .NOTES White-label — Intune Solutions. Default -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param() # Default to -WhatIf unless the caller explicitly turned it off if (-not $PSBoundParameters.ContainsKey('WhatIf') -and -not $PSBoundParameters.ContainsKey('Confirm')) { $WhatIfPreference = $true } Write-Host "=== Custom Compliance Remediation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan if ($WhatIfPreference) { Write-Host " (preview mode — pass -WhatIf:`$false to apply)" -ForegroundColor Yellow }

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS In preview, the actions it would take. When applied, an MDM sync is requested; re-run CUSTCOMP-Report.ps1 / check the portal after a policy cycle. .NOTES White-label — Intune Solutions. Default -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param() # Default to -WhatIf unless the caller explicitly turned it off if (-not $PSBoundParameters.ContainsKey('WhatIf') -and -not $PSBoundParameters.ContainsKey('Confirm')) { $WhatIfPreference = $true } Write-Host "=== Custom Compliance Remediation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan if ($WhatIfPreference) { Write-Host " (preview mode — pass -WhatIf:`$false to apply)" -ForegroundColor Yellow }

CUSTCOMP-Report.ps1

Purpose: Lists the custom compliance policies in Intune and reports each managed Windows device's overall compliance state and last sync, so you can see which devices are failing after a custom rule is assigned. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Report.ps1 -CsvPath C:\Temp\custcomp.csv Requires Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All + DeviceManagementConfiguration.Read.All. EXPECTED RESULTS The custom compliance policies found, a breakdown of devices by compliance state, and a per-device CSV (device, compliance state, last sync). .NOTES White-label — Intune Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All','DeviceManagementConfiguration.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Report.ps1 -CsvPath C:\Temp\custcomp.csv Requires Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All

+ DeviceManagementConfiguration.Read.All. EXPECTED RESULTS The custom compliance policies found, a breakdown of devices by compliance state, and a per-device CSV (device, compliance state, last sync). .NOTES White-label — Intune Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All','DeviceManagementConfiguration.Read.All' -NoWelcome

CUSTCOMP-Set-ScopeTargeting.ps1

Purpose: Ensures the scope groups used to stage custom compliance exist in Microsoft Entra ID: 'CustomCompliance - Devices - Pilot' and 'CustomCompliance - Devices - Broad'. Creates them if missing. Optionally adds a named device (by device name) to the pilot group. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Set-ScopeTargeting.ps1 ` [-PilotDeviceName "PC-001"] Requires Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All for -PilotDeviceName). EXPECTED RESULTS Both groups exist (created or already present); their object IDs are printed for assignment. .NOTES White-label — Intune Solutions. Creates groups when missing. #> [CmdletBinding(SupportsShouldProcess=$true)] param([string]$PilotDeviceName) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','Device.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Set-ScopeTargeting.ps1 ` [-PilotDeviceName "PC-001"] Requires Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All for -PilotDeviceName). EXPECTED RESULTS Both groups exist (created or already present); their object IDs are printed for assignment. .NOTES White-label — Intune Solutions. Creates groups when missing. #> [CmdletBinding(SupportsShouldProcess=$true)] param([string]$PilotDeviceName) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','Device.Read.All' -NoWelcome

CUSTCOMP-Validation.ps1

Purpose: Checks that the JSON rule file is valid JSON, that its Rules have the required fields, and that every rule SettingName is a property the detection script emits — the #1 cause of an Error state in custom compliance. Read-only: it does not change Intune or the device. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Validation.ps1 ` -ScriptPath .\detection.ps1 -RulePath .\rule.json EXPECTED RESULTS "PAIRING VALID" when the JSON parses, rules are well-formed, and setting names all match. .NOTES White-label — Intune Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ScriptPath, [Parameter(Mandatory)][string]$RulePath) Write-Host "=== Custom Compliance — script/rule validation ===" -ForegroundColor Cyan if (-not (Test-Path $ScriptPath)) { Write-Host "Script not found: $ScriptPath" -ForegroundColor Yellow; exit 1 } if (-not (Test-Path $RulePath)) { Write-Host "Rule not found: $RulePath" -ForegroundColor Yellow; exit 1 }

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMP-Validation.ps1 ` -ScriptPath .\detection.ps1 -RulePath .\rule.json EXPECTED RESULTS "PAIRING VALID" when the JSON parses, rules are well-formed, and setting names all match. .NOTES White-label — Intune Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ScriptPath, [Parameter(Mandatory)][string]$RulePath) Write-Host "=== Custom Compliance — script/rule validation

```
=== " -ForegroundColor Cyan if (-not (Test-Path $ScriptPath)) { Write-Host "Script not found: $ScriptPath"
-ForegroundColor Yellow; exit 1 } if (-not (Test-Path $RulePath)) { Write-Host "Rule not found: $RulePath"
-ForegroundColor Yellow; exit 1 }
```